

BeaglePlay Service Architecture & Operations Guide

Purpose: Understand the existing BeaglePlay Debian system before adding or modifying features.

Scope: This document is based on the supplied inventory log and the clarification that GNSS_Rover_Base.jar, gnss_gui_server, index.html, and the UI code were written by you.

Board software: Debian GNU/Linux 12 (Bookworm), ARM64, Linux 6.12.43-ti-arm64-r53 PREEMPT_RT.

1. The Most Important Architectural Idea

A Linux **service** is normally a systemd lifecycle definition. The service is not necessarily the application logic. It tells systemd which program to start, when to start it, which user should run it, what it depends on, and what to do if it exits.

For your own GNSS Java application the startup chain is conceptually:

```
systemd → gnss.service → /opt/gnss/start_gnss.sh → java -jar /home/debian/GNSS_Rover_Base.jar
```

For your GUI server:

```
systemd → gnss_gui_server.service → /home/debian/gnss_gui_server → HTTP/web resources such as index.html and ui/
```

Therefore, your application code and the service that launches it should be analyzed separately.

2. High-Level System Architecture

BeaglePlay hardware

UART / GPIO / Ethernet / Wi-Fi / USB / watchdog

↓

Debian + systemd

↓

Existing hardware/network services

↓

Your applications

GNSS_Rover_Base.jar + gnss_gui_server + web UI

This architecture allows additional functionality to be introduced as separate services, provided that shared serial ports, GPIOs, TCP/UDP ports, files, and network interfaces are not taken over incorrectly.

3. Your Applications and Their Service Managers

Your software	Linux manager	Role
GNSS_Rover_Base.jar	gnss.service	Starts and supervises the Java GNSS application
gnss_gui_server	gnss_gui_server.service	Starts and supervises your GUI/web server
index.html / ui/	Served/used by your GUI server	Web interface; not a systemd service itself

4. Custom Service Inventory

The supplied system inventory showed these important custom services. They should be treated as the main objects for reverse-engineering the product architecture.

Service	Primary purpose / current interpretation
gnss.service	Launches and keeps your GNSS Java application running.

Service	Primary purpose / current interpretation
gnss_gui_server.service	Launches your native GUI/web server.
gnss-watchdog.service	Serves the Linux hardware watchdog, using /dev/watchdog1; intended to reset/recover the board if software
gnss_gpio.service	GNSS-related GPIO control/read service; uses libgpiod and references gpiochip3 / gnss_pw_en.
serial-bridge.service	Serial-to-TCP bridge; binary references /dev/ttyS0 and TCP port 2000.
mcu_server.service	TCP-to-serial server for an MCU; binary references /dev/ttyS3.
ip-manager.service	Applies/configures the GNSS network IP settings and can restart gnss.service.
gnss-wifi-setup.service	Configures Wi-Fi AP settings, including SSID derived from device serial number.
gnss-wifi-connect.service	Attempts to connect wlan0 to an external Wi-Fi network for Internet access.
quectel-check.service	Checks the Quectel EC200U/eth2 connectivity and modem-related network state.
modem_init.service	Modem initialization support; needs detailed dependency/configuration review.
modem_status.service	Modem status monitoring; paired with a timer.
nginx.service	Web server/reverse proxy; exact relationship to your GUI server must be verified before changing it.
nodered.service	Node-RED runtime; separate application/automation environment.
docker.service / containerd.service	Container infrastructure; relevant only if containers are used by the product.
NetworkManager / systemd-networkd	Linux network management layers; do not modify casually because they affect the whole board.

5. Detailed Understanding of the Main Custom Services

5.1 gnss.service

This is the systemd entry point for your Java GNSS application. The service definition uses /opt/gnss/start_gnss.sh, a working directory under /opt/gnss, the debian user, and a restart policy. The important point is that systemd manages the process lifecycle; the GNSS business logic remains in your JAR.

Expected lifecycle: boot → systemd → gnss.service → start_gnss.sh → Java process. If the Java process exits and Restart=always remains configured, systemd attempts to start it again.

5.2 gnss_gui_server.service

This is the systemd entry point for your native GUI/web server. The server is separate from the Java application. Its web resources include your GUI files such as index.html and ui/. Treating the GUI server as a separate process reduces coupling: the GUI server can be restarted independently of the Java application, subject to whatever API/data dependencies your code has.

5.3 gnss-watchdog.service

The watchdog program uses /dev/watchdog1 and repeatedly services ('kicks') the hardware watchdog. The inventory showed recurring watchdog messages. The purpose is resilience: if the system becomes unresponsive and the watchdog is no longer serviced, the hardware watchdog can cause a reset. Do not reuse /dev/watchdog1 or change this service without understanding the board's watchdog configuration.

5.4 gnss_gpio.service

This service runs a custom GPIO program as root. The binary references libgpiod, gpiochip3, GPIO input/output functions, and the name gnss_pw_en. This indicates hardware control/readback associated with the GNSS subsystem. A new feature must not claim the same GPIO line without first mapping the exact line and consumer.

5.5 serial-bridge.service

The serial bridge provides a network-to-serial path. Evidence in the inventory includes /dev/ttyS0, TCP port 2000, tcp_to_serial, serial_reader, client_ip, and SERIAL_PORT. The working model is: TCP client ↔ serial_bridge ↔ /dev/ttyS0. Before adding a new serial feature, identify who owns ttyS0 and whether the bridge is bidirectional.

5.6 mcu_server.service

The MCU server is another serial/network bridge. It references /dev/ttyS3 and a TCP listening port. The working model is: TCP client ↔ mcu_server ↔ /dev/ttyS3 ↔ MCU. Do not open ttyS3 from a new application until its ownership and protocol are confirmed.

5.7 ip-manager.service

This utility reads GNSS IP configuration and applies an address to eth0. The inventory showed configured IP/port values and commands that add an IP address and restart gnss.service. Because it changes networking and restarts your GNSS application, it is an important coordination point.

5.8 Wi-Fi services

gnss-wifi-setup configures the AP side, including SSID behavior based on the device serial number. gnss-wifi-connect is intended to connect wlan0 to an external Wi-Fi network. These are different jobs: one is device/AP configuration, the other is client connectivity.

5.9 Quectel / modem services

quectel-check and modem-related services monitor or initialize the EC200U cellular modem and eth2. The inventory referenced ttyUSB devices, eth2, IP state, and modem status files/logs. These services must be considered before changing USB serial or cellular networking behavior.

6. Normal Debian Services

Services such as systemd-journald, systemd-udevd, dbus, systemd-logind, systemd-timesyncd, getty, polkit, cron, udisks2, SSH, Avahi, and similar components provide general operating-system functions. They are not part of your GNSS business logic. Avoid changing them unless a specific requirement demands it.

7. Critical Shared Resources

Before adding any new service, check these resources: • Serial devices: /dev/ttyS*, /dev/ttyUSB* • GPIO lines/chips • TCP/UDP ports • eth0 / eth2 / wlan0 • watchdog device /dev/watchdog1 • configuration files under /home/debian/gnss and /opt/gnss • logs • files consumed by more than one service • systemd targets/timers and scheduled reboots A new service should normally have a unique executable, configuration directory, log strategy, and network port unless it intentionally integrates with an existing interface.

8. How to Add a New Service Safely

Recommended architecture: create a separate application and a separate systemd unit. Do not edit gnss.service merely to add unrelated functionality.

Example pattern:

```
/opt/my-feature/my_feature.sh or my_feature binary
/opt/my-feature/config/
/opt/my-feature/logs/
/etc/systemd/system/my-feature.service
```

A typical unit concept is:

```
[Unit]
Description=My Feature Service
After=network-online.target

[Service]
Type=simple
User=debian
ExecStart=/opt/my-feature/my_feature
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

The exact dependencies, permissions, device access, and restart policy must be chosen from the feature's real requirements. Do not blindly copy this example into production.

9. Safe Change Rules

1. Never change a service until its current ExecStart, dependencies, devices, ports, and logs are recorded.
2. Never share a serial port casually.
3. Never reuse an occupied TCP/UDP port.
4. Avoid modifying NetworkManager/systemd-networkd unless necessary.
5. Keep new applications isolated under their own directory.
6. Give new services unique names.
7. Prefer Restart=on-failure for ordinary application services unless there is a reason for always restarting.
8. Test a new service manually before enabling it at boot.
9. Keep a rollback copy of every changed .service file and configuration file.
10. After changes, verify systemctl --failed and journalctl for errors.

10. Recommended Investigation Order

Phase 1 — Startup chain

Power-on → systemd → network/hardware → custom services → your Java app → your GUI server.

Phase 2 — Resource ownership

Map every UART, GPIO, TCP/UDP port, network interface, configuration file, and log file.

Phase 3 — Dependencies

Build a graph showing which services require or merely precede other services.

Phase 4 — Data flow

Trace GNSS data, RTCM/NTRIP, MCU data, modem traffic, GUI/API traffic, and files.

Phase 5 — Extension plan

Only after the above is understood should new services/features be designed.

11. Practical Service Inspection Commands

These commands are read-only and are useful whenever you need to understand a service:

```
systemctl status SERVICE_NAME
systemctl cat SERVICE_NAME
systemctl show SERVICE_NAME
systemctl list-dependencies SERVICE_NAME
journalctl -u SERVICE_NAME --no-pager
journalctl -u SERVICE_NAME -b --no-pager
systemctl is-enabled SERVICE_NAME
systemctl is-active SERVICE_NAME
```

For example:

```
systemctl cat gnss.service
systemctl status gnss.service
systemctl list-dependencies gnss.service
journalctl -u gnss.service -b --no-pager
```

12. Current Working Architecture

Power ON

↓

Linux kernel

↓

systemd

■■■ Network services

■■■ Hardware/device services

■■■ Wi-Fi / 4G services

■■■ GNSS GPIO / watchdog / serial services

■■■ gnss.service → **YOUR GNSS_Rover_Base.jar**

■■■ gnss_gui_server.service → **YOUR gnss_gui_server**

↓

your index.html / ui / GUI resources

13. Important Conclusions

The safest way to extend this board is not to modify the existing GNSS application service unless the new feature is intrinsically part of that application. Prefer a new, independently managed systemd service for independent functionality. The main risks are not systemd itself; they are shared hardware resources, shared network ports, shared configuration/data files, network-management conflicts, and unintended service dependencies.

14. Items Requiring Exact Verification Before Implementation

The inventory is sufficient for the architecture baseline, but the following should be verified from the live board before we make engineering decisions: exact ExecStart and dependency chains for every custom service; exact TCP/UDP listeners; physical UART-to-device mapping; GPIO line numbers and consumers; nginx-to-GUI relationship; Node-RED/Docker usage; scheduled timers/reboots; and the exact ownership of GNSS configuration/log files.

15. Recommended Next Engineering Step

The next document/update should be the **BeaglePlay Resource & Dependency Map**. It should contain one row per service with: service file, executable, user, startup trigger, restart policy, dependencies, serial devices, GPIOs, network interfaces, TCP/UDP ports, configuration files, log files, and impact if stopped. This is the document you should consult before adding any new service.